

Documentation Change Control and Delivery Tracking Procedure

Contents

Release statement.....	3
Documentation change control.....	3
Document lifecycle states.....	4
Initiate a change request.....	6
Prepare the change.....	6
Review and approve the change.....	7
Apply CUI markings.....	8
Deliver the document to the government Configuration Manager.....	9
Roles and responsibilities.....	10
Record formats.....	11
GitLab label set.....	12

Release statement

Read this statement before you read the remainder of this document.

Content and release condition

This document contains none of the following:

- Classified information of any level.
- Controlled Unclassified Information (CUI).
- Proprietary information of any company, or information that is subject to a non-disclosure agreement.
- Export-controlled technical data under the International Traffic in Arms Regulations (ITAR), 22 CFR 120-130.
- Export-controlled technical data under the Export Administration Regulations (EAR), 15 CFR 730-774.

All content is original work. The procedure comes from publicly available sources: the National Archives CUI rule at 32 CFR 2002, DoD Instruction 5200.48, the public CUI Registry, and the public GitLab documentation. The program names, contract numbers, document numbers, and personnel names in the examples do not identify a real contract or a real person.



Attention: This document shows one method. It is not the approved procedure for any contract. The contract Data Item Descriptions, the Contract Data Requirements List, and the Security Classification Guide have precedence over this document in all cases.

Purpose

This document is a technical writing sample. It shows the use of DITA topic types, hazard statements, tabular reference data, and Simplified Technical English in a process document.

Preparation

Prepared by Nathan B. Smith.

Documentation change control

Change control is the set of rules that makes each change to a document visible, approved, and recorded.

Why change control is necessary

A documentation deliverable is a contract item. The government accepts a specified document at a specified revision on a specified date. If the team cannot show which change went into which revision, the team cannot show that it satisfied the contract.

Change control gives three capabilities:

- **Traceability** - Each change in the document connects to a request, an approval, and a person.
- **Reversibility** - The team can go back to any previous revision.
- **Evidence** - The team can show an auditor what it delivered and when it delivered it.

What is under control

These items are under change control:

- The source files of the document.
- The graphics and the data files that the source files use.
- The stylesheets and the build configuration.

- The revision log and the delivery register.

Note: Put the source files under control, not the output files. The build makes the output files again from the source at any time. A PDF in the repository becomes incorrect at the first change to the source.

Two records, two purposes

The team keeps two records. Do not combine them:

- The **revision log** records the changes in one document. It is part of the document, and the reader sees it.
- The **delivery register** records the documents that went to the customer. It is a project record, and the customer does not receive it.

Refer to [Record formats](#) for the fields of each record.

The tools

This procedure uses GitLab for two functions:

- An **issue** holds the request for a change and the reason for it.
- A **merge request** holds the change itself, the review, and the approval.

The Git history gives the record of who changed each line and when. The Git tag gives the identity of the revision that went to the customer.

Related tasks

[Deliver the document to the government Configuration Manager](#) on page 9

The delivery is not complete when you send the file. The delivery is complete when the government Configuration Manager gives a receipt.

Related reference

[Record formats](#) on page 11

These tables give the fields of the revision log and the fields of the delivery register.

Document lifecycle states

A document is in one state at a time. The state shows what the team can do to the document and who has it.

The six states

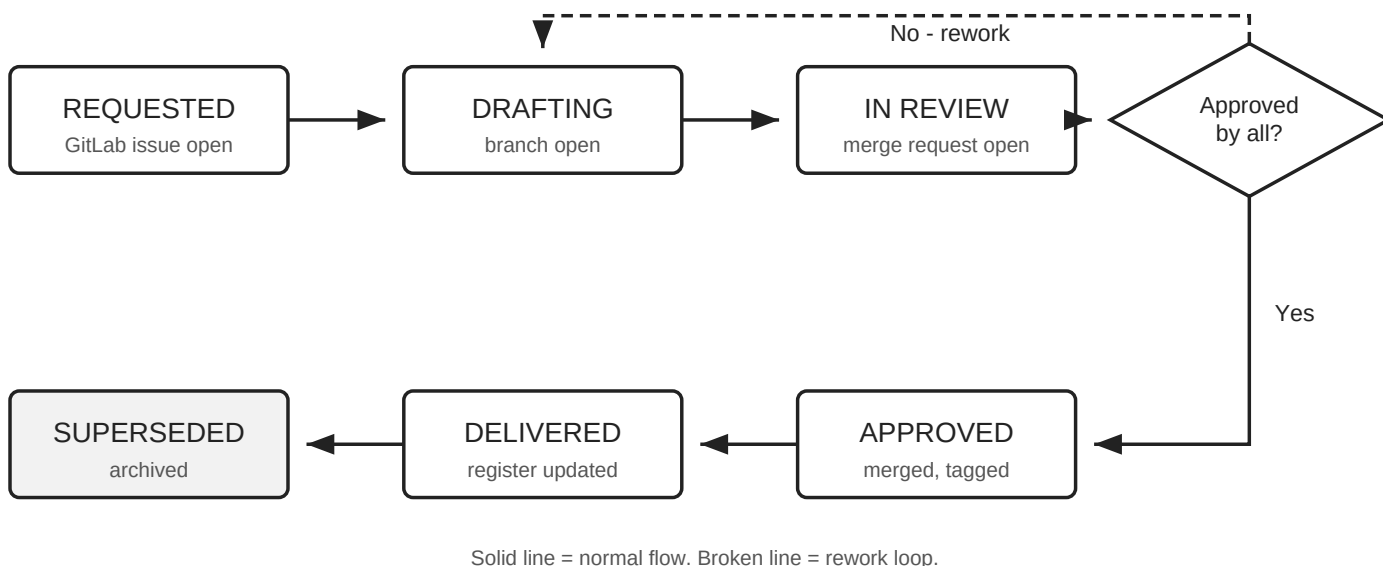


Figure 1: Lifecycle of a documentation change

- **Requested** - Someone found a problem or a new requirement. The GitLab issue is open. No source file has a change.
- **Drafting** - A writer has the change. The branch is open. The change is not visible in the main branch.
- **In review** - The merge request is open. The reviewers have the change. The writer makes no change to the scope in this state.
- **Approved** - All necessary approvals are complete. The branch is merged, and the commit has a tag.
- **Delivered** - The document went to the government Configuration Manager. The delivery register has the record.
- **Superseded** - A later revision replaced this revision. The revision stays in the repository, but no person uses it for work.

Rules for a change of state

These rules apply to each change of state:

1. A document goes to the next state only. It does not go forward more than one state at a time.
2. A document goes back to the Drafting state only. It does not go back more than one state at a time.
3. Each change of state has a record in the GitLab issue.



CAUTION: Do not make a change to a document that is in the In review state. A change during a review makes the review record incorrect, because the reviewers approved different content. If a change is necessary, close the merge request and go back to the Drafting state.

Related tasks

[Review and approve the change](#) on page 7

The merge request is the review record. It shows who approved the change and which content they approved.

Related reference

[GitLab label set](#) on page 12

Apply one type label and one priority label to each issue. The labels make the queue sortable.

Initiate a change request

Each change starts with a GitLab issue. The issue is the record of the reason for the change.

You must have Reporter permissions or higher on the documentation project.

Make an issue for each change, including a small change. An issue that has one line of text is sufficient for a spelling correction. A change that has no issue has no reason, and a person who does maintenance later cannot find why the text is different.

1. Open the documentation project in GitLab. Then select **Issues** > **New issue**.
2. Write the title in this form: [DOC-ID] short statement of the problem.

Example: [SUM-4471] Step 6 gives the wrong torque value.

3. Complete these fields in the description:

Field	Content
Document	The document number and the current revision.
Location	The topic identity or the section number.
Problem	What is incorrect, absent, or unclear.
Source	The engineering document, the test report, or the customer comment that shows the correct data.
Effect	What can occur if the document stays without a change.

4. Apply one type label and one priority label.

Refer to [GitLab label set](#).



Attention: A change that has an effect on safety gets the `prio::critical` label. Tell the responsible engineer on the same day. Do not wait for the next status meeting.

5. Attach the source data to the issue.



Warning: Do not attach a file that has CUI markings or export-controlled data to an issue in a repository that persons without authorization can read. Give the document number and the location of the file in the approved system in place of the file. A file in a repository is available to each person who has access to the project.

6. Select **Create issue**.

GitLab gives an issue number. Use this number in the branch name and in the revision log.

The change request has a number, a reason, and a priority.

Do the procedure [Prepare the change](#).

Prepare the change

Make the change on a branch. A branch keeps the incomplete work away from the approved content.

Before you start, make sure that:

- The change request has an issue number.
- You have Developer permissions on the project.
- You have the approved source data for the new content.



Warning: Write only from released engineering data. Do not write a procedure from a spoken statement, from a preliminary drawing, or from your memory. An incorrect maintenance procedure can cause an injury to a person and damage to equipment.

1. Get the current main branch. Type these commands:

```
git checkout main
git pull
```

2. Make a branch. Use this name form: doc/<issue-number>-<short-name>.

```
git checkout -b doc/482-sum4471-torque-value
```

Note: The issue number in the branch name lets GitLab connect the branch, the merge request, and the issue without more work.

3. Make the change to the source files.

Change the smallest quantity of content that corrects the problem. Do not correct other problems on the same branch. A second problem gets a second issue.

4. Add a line to the revision log of the document.

Refer to [Record formats](#) for the fields.

5. Build the output and examine it.

```
dita -i document.ditamap -f pdf2 -o out/
```



CAUTION: Examine the built output, not the source only. A change that is correct in the source can make an incorrect page break, an incorrect cross-reference, or an incorrect table. The customer receives the output.

6. Commit the change. Use this message form:

```
git commit -m "SUM-4471: correct torque value in step 6

Source: Engineering Change Notice ECN-2026-0331.
Closes #482"
```

The first line gives what changed. The body gives the source data. The last line closes the issue at the merge.

7. Send the branch to the server. Type this command:

```
git push -u origin doc/482-sum4471-torque-value
```

The change is on the server, but it is not in the main branch. The approved content has no change.

Do the procedure [Review and approve the change](#).

Review and approve the change

The merge request is the review record. It shows who approved the change and which content they approved.

The branch must be on the server, and the built output must have no error.

Two approvals are necessary for each change: an editorial approval and a technical approval. One person does not give both approvals. Refer to [Roles and responsibilities](#).

1. Open a merge request from your branch to the main branch.
2. Attach the built output file to the merge request.

The technical reviewer reads the output, not the XML source. The attachment decreases the time of the review.

3. Add the editorial reviewer and the technical reviewer as approvers.



Attention: The technical reviewer must be the engineer who is responsible for the system, or a person that the engineer identifies in writing. A different writer is not a technical reviewer.

4. Wait for the reviewers. Answer each comment.

Make a change or give a reason for no change. Then set the comment thread to resolved. Do not resolve a thread that has no answer.

5. Required: Make sure that both approvals are visible in the merge request before the merge.



Warning: Do not merge a change that does not have both approvals. An unapproved change in a delivered document is a contract nonconformance. It can also put incorrect instructions in the hands of a maintainer.

6. Merge the branch. Then delete the branch.

7. Put a tag on the merge commit. Use this form:

```
git tag -a SUM-4471-revC -m "SUM-4471 Revision C, approved 2026-07-25"
git push origin SUM-4471-revC
```

Note: The tag makes the revision findable one year later. Without a tag, a person must find the correct commit in the history.

The change is in the main branch. The merge request holds the record of the two approvals. The tag identifies the revision.

If the document goes to the customer, do the procedure [Apply CUI markings](#), and then the procedure [Deliver the document to the government Configuration Manager](#).

Related concepts

[Document lifecycle states](#) on page 4

A document is in one state at a time. The state shows what the team can do to the document and who has it.

Related reference

[GitLab label set](#) on page 12

Apply one type label and one priority label to each issue. The labels make the queue sortable.

Apply CUI markings

A document that contains Controlled Unclassified Information must have a banner marking and a designation indicator before the delivery.

Before you start, make sure that:

- The government sponsor identified the CUI category of the content in writing.
- You have the current Security Classification Guide for the program.
- You completed the CUI training that the contract specifies.



Warning: Do not decide the CUI category by yourself. Only the government authority makes this decision. An incorrect marking can cause an unauthorized release, or it can stop a legitimate release. If you have a doubt, ask the government sponsor before the delivery.

The marking has two parts. The banner marking goes on each page. The designation indicator goes on the first page only. The public CUI Registry gives the approved category markings.

1. Put the banner marking at the top center and at the bottom center of each page.

The banner has this form:

CUI

If the content has a specified category and a dissemination control, the banner has this form:

CUI//SP-CTI//FEDCON

Note: Put the banner in the page master of the stylesheet, not in the content of a topic. A banner in the content is absent from the first page or is present two times on a page.

- Put the designation indicator on the first page. Include these four items:

Item	Content
Controlled by	The government organization that controls the information.
Controlled by (office)	The office in that organization.
Category	The category marking from the CUI Registry.
POC	The name or the office telephone number of the point of contact.

- Add the distribution statement that the contract specifies.



CAUTION: The distribution statement and the CUI banner are different controls. A document needs both. Do not use one in place of the other.

- Build the output. Then examine the first page, one middle page, and the last page.

The banner is at the top and at the bottom of each page. The designation indicator is on the first page.

- Record the marking review in the delivery register.

The document has the correct markings for a CUI delivery.

Deliver the document to the government Configuration Manager

The delivery is not complete when you send the file. The delivery is complete when the government Configuration Manager gives a receipt.

Before you start, make sure that:

- The merge request shows both approvals.
- The commit has a tag.
- The markings are correct.
- You know the Contract Data Requirements List (CDRL) item number and the delivery date that the contract specifies.

- Build the output from the tagged commit. Type these commands:

```
git checkout SUM-4471-revC
dita -i document.ditamap -f pdf2 -o delivery/
```



CAUTION: Build from the tag, not from the main branch. The main branch can contain a later change that has no approval for this delivery.

- Give the file this name form: <doc-number>_<revision>_<YYYYMMDD>.pdf.

Example: SUM-4471_RevC_20260725.pdf.

- Make the transmittal letter. Include the document number, the revision, the CDRL item number, the contract number, the date, and the list of changes from the previous revision.

4. Required: Send the file by the method that the contract specifies.



Warning: Send CUI by the approved method only. Do not send CUI in an unencrypted electronic mail message, and do not put CUI in a commercial file sharing service. An unauthorized release is a reportable security incident.

5. Add a line to the delivery register on the same day.

Refer to [Record formats](#). Leave the receipt fields empty at this time.

6. Ask the government Configuration Manager for a receipt.

7. When the receipt comes, write the receipt date and the name of the receiver in the delivery register. Then attach the receipt to the GitLab issue.



Attention: If no receipt comes in five work days, ask again and record the second request. A delivery that has no receipt is a delivery that the customer can dispute.

8. Set the state of the previous revision to Superseded in the delivery register.

The customer has the document. The delivery register shows the date, the revision, and the receipt. The team can show the delivery to an auditor.

Related concepts

[Documentation change control](#) on page 3

Change control is the set of rules that makes each change to a document visible, approved, and recorded.

Related reference

[Record formats](#) on page 11

These tables give the fields of the revision log and the fields of the delivery register.

Roles and responsibilities

This table gives the role of each person in the change control process.

Table 1: Roles

Role	What the person does	What only this role can do
Author	Writes the change on a branch and keeps the revision log.	Nothing. Each action has a review.
Editorial reviewer	Examines the language, the structure, and the style guide compliance.	Gives the editorial approval.
Technical reviewer	Examines the technical content against the released engineering data.	Gives the technical approval.
Contractor Configuration Manager	Keeps the delivery register and controls the tags.	Approves a change to the branch rules and the release process.
Government Configuration Manager	Receives the delivery and gives the receipt.	Accepts a contract deliverable.
Information System Security Officer	Gives the rules for the handling of controlled information.	Approves a change to the marking and the transmission method.



Attention: One person can hold more than one role, but one person cannot be the author and a reviewer of the same change. A review by the author is not a review.

Record formats

These tables give the fields of the revision log and the fields of the delivery register.

Revision log

The revision log is in the document. The reader sees it. Add one line for each approved change.

Table 2: Revision log fields

Field	Content	Example
Revision	The revision letter or number.	C
Date	The date of the approval, not the date of the writing.	2026-07-25
Sections	The topics or the sections that have a change.	Section 4.2, step 6
Description	What changed, in one sentence. Write it for the reader, not for the writer.	Corrected the torque value.
Authority	The engineering document that permits the change.	ECN-2026-0331
Issue	The GitLab issue number.	#482

Note: Write the description for the maintainer. A line that says "various corrections" gives no help to the person who must know if this revision has an effect on the work.

Delivery register

The delivery register is a project record. The customer does not receive it. Keep one line for each delivery.

Table 3: Delivery register fields

Field	Content	Example
Document	The document number.	SUM-4471
Revision	The revision that went to the customer.	C
Tag	The Git tag of the delivered content.	SUM-4471-revC
CDRL item	The Contract Data Requirements List item number.	A004
Date due	The date that the contract specifies.	2026-07-31
Date sent	The date of the transmission.	2026-07-25
Method	The approved transmission method.	DoD SAFE
Markings	The banner marking of the delivered file.	CUI//SP-CTI
Receipt date	The date of the receipt from the government.	2026-07-28
Received by	The name of the person who gave the receipt.	Government CM
State	Current or Superseded.	Current



CAUTION: The register is the only record that connects a contract deliverable to the content that went to the customer. Keep it in a location that has a backup. A register on one workstation is not a project record.

Related concepts

[Documentation change control](#) on page 3

Change control is the set of rules that makes each change to a document visible, approved, and recorded.

Related tasks

[Deliver the document to the government Configuration Manager](#) on page 9

The delivery is not complete when you send the file. The delivery is complete when the government Configuration Manager gives a receipt.

GitLab label set

Apply one type label and one priority label to each issue. The labels make the queue sortable.

Type labels

Table 4: Type labels

Label	What it actually means	Use it when
<code>type::error</code>	The document says something that is not true.	A value, a part number, or a step is wrong.
<code>type::gap</code>	The document does not say something that it must say.	A procedure has no warning, or a task is absent.
<code>type::clarity</code>	The document is true, but the reader cannot understand it.	A step has two meanings, or a sentence is too long.
<code>type::engineering-change</code>	The hardware or the software changed, and the document must follow.	An engineering change notice arrives.
<code>type::style</code>	The content is correct, but it does not obey the style guide.	Terminology, capitalization, or format is not consistent.
<code>type::toolchain</code>	The build, the stylesheet, or the output has a problem.	A page break or a cross-reference is incorrect.

Priority labels

Table 5: Priority labels

Label	What it actually means	Response time
<code>prio::critical</code>	A person can be hurt, or equipment can be damaged, if the document stays without a change.	Same day
<code>prio::high</code>	The next delivery is not correct without this change.	Before the next delivery
<code>prio::normal</code>	The change is necessary, but it can wait for the normal cycle.	Next revision
<code>prio::low</code>	The change makes the document better, but no person will fail without it.	When time is available



Warning: A `prio::critical` issue needs immediate notification to the responsible engineer and to the program manager. A label in a tracking system is not a notification. Speak to the person.

State labels

GitLab sets these labels automatically when the connected merge request changes state. Do not set them manually.

- `state::drafting` - A branch is open for this issue.
- `state::in-review` - A merge request is open.
- `state::approved` - The merge request has all approvals.
- `state::delivered` - The delivery register has the record.

Related concepts

[Document lifecycle states](#) on page 4

A document is in one state at a time. The state shows what the team can do to the document and who has it.

Related tasks

[Review and approve the change](#) on page 7

The merge request is the review record. It shows who approved the change and which content they approved.